UNIVERSITÉ DE MONTPELLIER

Faculté des Sciences
Master 1 Informatique – Génie logiciel

HAI716 – Systèmes multi-agents et programmation orientée agent

Projet 1

Robot de réconfort

Auteurs	Mohammed M'BARKI et Mohamed MESTI
Numéros étudiants	2261564 et 22616577
Parcours	Génie logiciel – GL
Année universitaire	2026–2027

Compte rendu de projet

Table des matières

1 Introduction **1**

2 Architecture générale **1**

2.1 Organisation des composants 1

2.2 Répartition des responsabilités 1

3 Méthode **1**

3.1 Cycle de traitement 2

3.2 Exemple d'implémentation 2

4 Résultats **2**

5 Discussion **2**

6 Conclusion **3**

1 Introduction

Un système autonome reçoit des informations de son environnement, construit une représentation interne et choisit une action adaptée à son objectif. La qualité de son comportement dépend de la séparation entre ces responsabilités et de la cohérence des informations échangées.

Ce travail s'intéresse à l'organisation d'un tel système, à sa stratégie de décision et aux mécanismes utilisés pour contrôler son exécution.

Point important

L'agent ne prend ses décisions qu'à partir des informations auxquelles il a effectivement accès.

2 Architecture générale

2.1 Organisation des composants

L'architecture distingue l'acquisition des données, leur traitement et la production d'une action. Cette séparation limite les dépendances et rend chaque partie plus facile à tester.



Figure 1 – *Organisation générale du traitement.*

2.2 Répartition des responsabilités

Table 1 – *Responsabilités des principaux composants.*

Composant	Responsabilité	Données manipulées
Environnement	Produire les observations disponibles	État observable
Mémoire	Conserver les informations acquises	Représentation interne
Décision	Sélectionner la prochaine opération	Objectif et connaissances
Action	Modifier l'état du système	Commande autorisée

Remarque

Une frontière claire entre l'environnement et l'agent évite qu'une décision utilise directement une information non observée.

3 Méthode

3.1 Cycle de traitement

Le comportement suit un cycle simple et répétable :

1. recevoir une observation ;
2. mettre à jour la représentation interne ;
3. choisir une action autorisée ;
4. exécuter l'action et observer son résultat.

3.2 Exemple d'implémentation

Listing 1 – *Structure simplifiée d'un cycle de décision.*

```python
def executer_cycle(agent, environnement):
    observation = environnement.percevoir()
    agent.mettre_a_jour(observation)
    action = agent.decider()
    return environnement.executer(action)
```

Avertissement

Une action invalide doit être refusée sans modifier l'état de l'environnement.

4 Résultats

Les essais portent sur plusieurs situations représentatives. Chaque cas vérifie le résultat obtenu ainsi que la cohérence de la séquence d'actions produite.

Table 2 – *Synthèse des essais.*

Cas étudié	État	Observation
Parcours nominal	Validé	Objectif atteint
Information absente	Validé	Échec signalé proprement
Action impossible	Validé	État conservé

Les contrôles confirment la séparation des responsabilités et la stabilité du comportement dans les situations prises en compte.

5 Discussion

La solution privilégie une organisation explicite et un comportement déterministe. Cette approche facilite la compréhension et la reproduction des résultats. Elle peut cependant augmenter le nombre d'opérations lorsque de nouvelles informations imposent de recalculer une décision.

6 Conclusion

L'organisation proposée fournit une base claire pour construire, observer et tester le comportement d'un agent autonome. Les améliorations futures peuvent porter sur le coût de la décision, la gestion de l'incertitude et la qualité des stratégies de repli.